STUDY PROJECT — PHASE 3

Operational Complexity Meter

Quantifying Operational Complexity in Distributed Systems

Aabid Ali Sofi | 2023EBCS041
Advisor: Preethy P Johny | Feb 2026

Agenda

1. Problem Statement
2. What is OCM?
3. The 6 Metrics
4. System Architecture
5. Pipeline Deep Dive
6. Parser & Extraction
7. Graph Algorithms
8. Storage & API
9. Dashboard
10. Demo & Results
11. Future Work

SECTION

Problem Statement

The Problem

Modern distributed systems are **operationally complex**, but that complexity is rarely measured.

Symptoms:

- Hundreds of config parameters
- Deep dependency chains
- Frequent config changes
- Env-specific drift (dev vs prod)
- Exposed failure surfaces

Impact:

- Outages from config errors
- Slow incident response
- Hidden coupling between services
- Unmeasured toil & risk

Can we quantify operational complexity as a single, measurable score?

SECTION

What is OCM?

Operational Complexity Meter

OCM is a **single Go binary** that:

1. **Scans** a Kubernetes YAML repository
2. **Identifies** services from directory structure or manifest metadata
3. **Computes** 6 operational complexity metrics per service
4. **Normalizes** and produces a composite OCM score (0–1)
5. **Stores** results in SQLite for time-series tracking
6. **Serves** an interactive web dashboard

```
go run ./cmd/ocm --repo /path/to/k8s-repo --db ocm.sqlite --port 8080
```

No external dependencies. No K8s SDK. No CGO. Pure Go.

SECTION

The 6 Metrics

Metric Overview

Metric	What It Measures	How
CSA	Configuration Surface Area	Count of env vars, ports, resources, replicas, spec keys
DD	Dependency Depth	Longest path in the SCC-condensed dependency DAG
DB	Dependency Breadth	Direct upstream + downstream dependency count
CV	Change Volatility	Git commits affecting config in the last 30 days
FE	Failure Exposure	Exposed endpoints (LB, NodePort, Ingress, hostPort, ext URLs)

CDR	Config Drift Risk	Env-specific overrides (dev/staging/prod directory diffs)

Composite OCM Score

Normalization — Min-max across the service cohort:

$$\text{Normalized}(M) = \frac{M - M_{\text{min}}}{M_{\text{max}} - M_{\text{min}}}$$

Composite Score — Weighted sum of normalized metrics:

$$\text{OCM} = \frac{1}{6} \cdot \text{CSA} + \frac{1}{6} \cdot \text{DD} + \frac{1}{6} \cdot \text{DB} + \frac{1}{6} \cdot \text{CV} + \frac{1}{6} \cdot \text{FE} + \frac{1}{6} \cdot \text{CDR}$$

- Output range: **0.0** (minimal complexity) to **1.0** (maximum in cohort)
- Weights are **configurable** (default: equal 1/6 each)

- If max = min for a metric, normalized value = 0 (no variance)

SECTION

System Architecture

Single-Binary Architecture

```
┌──────────────────────────────────────────────────────────┐
│                      ocm (single binary)                 │
│                                                          │
│  ┌──────────┐   ┌──────────┐   ┌──────────────────┐     │
│  │ CLI/Main │──▶│ Pipeline │──▶│  SQLite Storage  │     │
│  └──────────┘   │          │   └──────────────────┘     │
│                 │ Parser   │            │               │
│                 │ GitLog   │            ▼               │
│                 │ Metrics  │   ┌──────────────────┐     │
│                 │ Scoring  │   │  HTTP API Server │     │
│                 └──────────┘   │  + Embedded      │     │
│                                │    Dashboard     │     │
│                                └──────────────────┘     │
└──────────────────────────────────────────────────────────┘
```

Only 2 dependencies:

- `yaml.v3`
- `modernc.org/sqlite`

No CGO required

Pure Go SQLite driver
Cross-compilable

Embedded assets

`go:embed` for HTML/CSS/JS
Zero external files

Data Flow Pipeline

1. **Discovery** — Walk filesystem, collect `.yaml`/`.yml`, skip `.git`/`vendor`
2. **Parsing** — Multi-doc YAML decode, extract CSA facts + deps + FE signals
3. **Service ID** — Map files to services via `dir` (first path component) or `manifest` (metadata.name)
4. **Metric Engine** — Compute DD (graph), DB (degree), CV (git), FE (exposure), CDR (drift)
5. **Normalization** — Min-max across cohort for each metric
6. **Composite** — Weighted sum ⟶ single OCM score per service
7. **Persist** — Atomic transaction: services + metrics + evidence + scores ⟶ SQLite
8. **Serve** — HTTP API + embedded dashboard at `localhost:8080`

SECTION

Parser & Extraction

CSA Extraction

For each Kubernetes manifest, count configuration surface contributors:

Component	Source	Count
`env`	Container environment variables	1 per var
`port`	Container port entries	1 per port
`resource`	Resource limits/requests leaf keys	1 per key
`replica`	`spec.replicas` field	1 if present
`spec_key`	Top-level keys under `spec`	1 per key

Each contributor is stored as an **evidence item** with source path, manifest kind, and manifest name for full traceability.

Dependency Extraction

Three heuristic detection rules for inter-service dependencies:

Rule 1 — Env var suffix matching: `_SERVICE`, `_SERVICE_HOST`, `_SERVICE_PORT`, `_ADDR`, `_HOST`, `_URL`, `_ENDPOINT`, `_URI`

Rule 2 — Cluster FQDN detection: Values containing `.svc.cluster.local` ⟶ extract service name

Rule 3 — Hostname:port pattern: Regex: `^[a-z][a-z0-9-]{0,62}:\d{2,5}$` ⟶ extract hostname

Designed for real-world repos: Sock Shop, Google Online Boutique, Helm charts, Rails/Django/Spring deployments.

Failure Exposure Detection

FE counts exposed endpoints and external integrations — 5 detection rules:

1. **LoadBalancer / NodePort** service types — each port = 1 endpoint
2. **Ingress** resources — each `rules[].http.paths[]` = 1 endpoint
3. **ExternalName** services — 1 external integration
4. **Container hostPort** — each hostPort = 1 endpoint
5. **External URLs** in env vars — `http(s)://` not containing `.svc.cluster.local`

Detection rules are explicitly documented in code per spec requirement.

SECTION

Graph Algorithms

Dependency Depth — Tarjan's SCC

DD = longest path (edge count) in the dependency graph.

Problem: Dependency graphs may have cycles (A ⟶ B ⟶ A).

Solution: SCC (Strongly Connected Components) condensation:

Step 1: Tarjan's algorithm finds SCCs
Step 2: Condense each SCC to a single node
Step 3: Result is a DAG (no cycles)
Step 4: DFS with memoization ⟶ longest path

Example:

```
A → B → C    (B ↔ A = cycle)
    ↑ ↙
    A

SCC: {A, B} → node S₁
DAG: S₁ → C
```

```
DD[A] = DD[B] = 1
DD[C] = 0
```

Algorithm is deterministic — same input always produces same DD values.

SECTION

Storage & API

SQLite Schema

4 tables, atomic transactions, time-series support:

Core tables:

```
services(id, name, repository)
  UNIQUE(name, repository)

metrics(id, service_id,
  metric_type, metric_value,
  timestamp)

composite_scores(id, service_id,
  ocm_score, timestamp)
```

Evidence table:

```
metric_evidence(id, service_id,
  metric_type, component,
  evidence_key, evidence_value,
  source_path,
  manifest_kind, manifest_name,
  timestamp)
```

- FK constraints with `PRAGMA foreign_keys`
- Indexes on `(service_id, metric_type, timestamp)`
- Upsert for idempotent re-runs

REST API

Endpoint	Returns
`GET /api/healthz`	Health check
`GET /api/services`	List all analyzed services
`GET /api/overview`	Repo aggregates + per-service summaries
`GET /api/services/{id}/scores`	OCM score time series
`GET /api/services/{id}/metrics/{type}`	Metric time series
`GET /api/services/{id}/metrics/{type}/evidence`	Evidence items for latest run

- Built with Go 1.22 `http.ServeMux` (method + path pattern matching)
- All responses are JSON with proper `Content-Type`

- CORS enabled for local development

SECTION

Dashboard

Dashboard Design

Stripe-inspired dark theme with responsive layout:

Repo Overview:
- Composite OCM average across all services
- 6 metric tiles with aggregated values
- Service count and API status

Service Overview:
- Individual service metrics
- OCM score trend chart (Canvas)
- Radar chart (normalized profile)
- Bar chart (raw values)
- Sparklines in metric tiles

Interactive features:
- Sidebar service selector
- "All Services" ↔ single-service toggle
- Evidence drill-down modal per metric
- Service comparison table
- Responsive (mobile, tablet, desktop)

Tech:
- Pure HTML/CSS/JS (no frameworks)
- Canvas-based charts
- Embedded via `go:embed`

Evidence-Based Traceability

Every metric contribution is tracked with provenance:

Metric	Evidence Type	Example
CSA	`env`, `port`, `resource`, `replica`	`PORT=8080` from `api/deploy.yaml`
DD	`dep_chain`, `direct_dep`	`api → cache → db` (depth=2)
DB	`upstream_dep`, `downstream_dep`	`orders → payment` edge
CV	`commit`	`commit abc12345`
FE	`loadbalancer`, `ingress`, `external_url`	LoadBalancer port 80
CDR	`env_override`	`config.yaml` in 2 envs: dev, prod

Users can click any metric tile → modal shows exactly **what** contributed to the score and **where** it was found.

SECTION

Testing & Validation

Test Suite

Pipeline tests:

- DD cycle handling (Tarjan's SCC)
- DB in/out degree counting
- Self-dependency ignored
- Normalization edge cases (max = min)
- All 6 metrics present in output
- Composite score in [0, 1]
- DD evidence generation

Parser tests:

- 7 dependency extraction tests
- 6 FE detection tests
- Suffix, FQDN, hostname:port patterns
- Multi-document YAML

Storage tests:

- Round-trip for all 6 metric types
- Empty source path handling

Git log tests:

- 6 tests for log parsing

All 29 tests pass · Build clean · Vet clean

Project Structure

```
ocm/
├── cmd/ocm/main.go        # CLI entry
├── internal/
│   ├── model/model.go     # Types
│   ├── parser/
│   │   ├── yamlscan.go    # K8s parser
│   │   └── parser_test.go
│   ├── pipeline/
│   │   ├── pipeline.go    # Orchestration
│   │   └── pipeline_test.go
│   ├── gitlog/
│   │   ├── gitlog.go      # CV metric
│   │   └── gitlog_test.go
│   ├── storage/
│   │   ├── storage.go     # SQLite layer
│   │   └── storage_test.go
│   ├── api/api.go         # HTTP API
│   └── dashboard/
│       ├── dashboard.go   # go:embed
│       └── static/
│           ├── index.html
│           ├── styles.css
│           └── app.js
├── specs/                  # 8 spec docs
├── docs/                   # Phase docs
└── go.mod                  # 2 deps
```

SECTION

Project Evolution

Phase 2 → Phase 3

Phase 2 (MVP/PoC):

- 2 metrics: CSA + DD only
- Basic dependency extraction
- Simple dashboard
- CSA evidence only
- Composite: 50/50 weights
- Manual validation

Phase 3 (Current):

- All 6 metrics: CSA, DD, DB, CV, FE, CDR
- Broadened dependency detection (9 patterns)
- Git integration for CV
- FE + CDR detection rules
- Evidence for all 6 metrics
- Stripe-inspired dashboard redesign
- Repo-wide + service-specific views
- Radar, bar, trend, sparkline charts
- 29 automated tests
- `--cv-window` CLI flag
- Composite: equal 1/6 weights

SECTION

Demo & Results

Running OCM

Analyze a Kubernetes repository:

```
# Build and run
go run ./cmd/ocm \
  --repo /path/to/microservices-demo \
  --db ocm.sqlite \
  --port 8080

# Output
ocm: analyzed 12 services, db=ocm.sqlite
ocm: listening on http://127.0.0.1:8080
```

JSON output mode (no server):

```
go run ./cmd/ocm --repo ./repo --print 2>/dev/null | jq '.services[].name'
```

Configure analysis:

```
--service-key manifest    # Use metadata.name instead of directory
--cv-window 60            # 60-day lookback for Change Volatility
```

Known Limitations

By design (prototype scope):

- Heuristic-based extraction
- No Helm template rendering
- No Dockerfile parsing
- Single-run-on-startup model
- `git` binary required for CV
- Localhost only (no auth)

Mitigation strategies:

- Evidence trail enables manual verification
- Multiple service ID strategies
- Best-effort approach (warnings, not failures)
- Modular architecture enables extension
- Configurable weights for composite scoring

SECTION

Future Work

Potential Extensions

Near-term:

- Helm v3 template rendering
- Dockerfile complexity parsing
- Scheduled/on-demand re-analysis
- Configurable weights via UI
- Export reports (PDF/CSV)

Long-term:

- CI/CD integration (PR complexity diffs)
- Multi-repo support
- Complexity budget enforcement
- Runtime metrics correlation
- Team/org-level dashboards
- Authentication & multi-user

Summary

- ✓ **6 metrics** quantifying operational complexity from K8s YAML
- ✓ **Single binary** — Go + embedded SQLite + embedded dashboard
- ✓ **Evidence-based** — every metric contribution is traceable
- ✓ **Graph algorithms** — Tarjan's SCC for cycle-safe dependency depth
- ✓ **Minimal dependencies** — only `yaml.v3` + pure-Go SQLite
- ✓ **Interactive dashboard** — repo overview, service drill-down, charts
- ✓ **29 automated tests** — parser, pipeline, storage, git log

Thank You

Questions?

Aabid Ali Sofi | 2023EBCS041
Operational Complexity Meter — Phase 3